Indexing, reuse, and reduction

Entries 1–9 of 27 · Passes, analyses, policies, and lowering mechanisms

P01
Symbolic index and effect analysis
Prove index equivalence, stride, variable use, and effects.
`compiler/analysis.py`

P02
Storage typing
Keep half storage separate from explicit FP32 arithmetic.
`compiler/storage.py`

P03
Elementwise vectorization
Turn scalar thread-strided loops into four values per lane.
`compiler/passes/vectorize.py`

P04
Reduction load forwarding
Reuse reduction inputs in registers in the following consumer.
`compiler/passes/regforward.py`

P05
Recurrence-axis reordering
Move score-independent tile axes into the lane-sharing position.
`compiler/passes/recurrence.py`

P06
Split-K grid folding
Merge split-K partitions into deeper per-CTA contractions.
`compiler/passes/gridfold.py`

P07
Cooperative dot reduction
Share a repeated dot product across a power-of-two lane group.
`compiler/transforms.py`

P08
Coalesced sum/square reduction
Coalesce sum/square loads and pipeline long register reductions.
`compiler/transforms.py`

P09
Small-tree elimination
Replace small shared reduction trees with direct loads and adds.
`compiler/passes/tailtree.py`

Tensor and recurrence mechanisms

Entries 10–18 of 27 · Passes, analyses, policies, and lowering mechanisms

P10
Warp tree specialization

Let warp 0 finish a warp-partial tree and broadcast the result.

`compiler/codegen/lower.py`

P11
Contraction tensorization

Recognize dense shared-memory contractions for compensated MMA.

`compiler/passes/contraction.py`

P12
Tensor shared-memory layout

Match tensor producers and consumers with bank-aware layouts.

`compiler/codegen/tensor.py`

P13
Packed register staging

Replace a certified copy ring with register prefetch and packed data.

`compiler/passes/regstage.py`

P14
Tensor fragment pipelining

Load the next MMA fragment before consuming the current fragment.

`compiler/codegen/tensor.py`

P15
Pipeline drain

Skip the final unused prefetch and consumer barrier.

`compiler/passes/regstage.py`

P16
Staged online recurrence

Stage recognized score/max/mass/value recurrences into GPU phases.

`compiler/passes/recurrence.py`

P17
Blocked recurrence loads and address hoisting

Group contiguous key rounds and hoist affine value addresses.

`compiler/passes/recurrence.py`

P18
Warp-distributed vector PV reduction

Distribute PV keys across four groups and vectorize four columns.

`compiler/passes/recurrence.py`

Normalization and final lowering

Entries 19–27 of 27 · Passes, analyses, policies, and lowering mechanisms

P19

Recurrence shared-storage reuse

Reuse dead staged-operand storage for shared probabilities.

`compiler/passes/lifetime.py`

P20

Hybrid row normalizer

Separate row maximum work from owned-value exponentials and mass.

`compiler/passes/hybrid_normalizer.py`

P21

Ordered hardware warp maximum

Use integer warp redux for proven non-NaN FP32 maximum.

`compiler/codegen/lower.py`

P22

CTA/thread address factoring

Decode CTA axes and thread axes separately.

`compiler/codegen/lower.py`

P23

Loop cost policy

Choose structure-sensitive LLVM unroll and vectorization hints.

`compiler/transforms.py`

P24

Reciprocal-square-root folding

Recognize adjacent sqrt-to-reciprocal dataflow for approximate rsqrt.

`compiler/transforms.py`

P25

Dead atomic-result elimination

Use a PTX reduction when an atomic add result is unused.

`compiler/ptx.py`

P26

Fixed block contract

Express the fixed launch width as a PTX block-size requirement.

`compiler/ptx.py`

P27

Approximate division cleanup

Select approximate FP32 division only when enabled.

`compiler/ptx.py`